

Capítulo 2

Gramáticas de concatenación de rango y el SAT como lenguaje formal

En este capítulo se presentan las gramáticas de concatenación de rango, formalismo de base sobre el que se desarrolla el resto del trabajo. Se analizan sus principales propiedades y limitaciones, y se revisan los antecedentes directos que abordan el problema de la satisfacibilidad booleana mediante formalismos de la teoría de lenguajes formales, con énfasis en los trabajos previos realizados en la Facultad de Matemática y Computación de la Universidad de La Habana. Los conceptos básicos utilizados en este capítulo se presentaron en el capítulo anterior.

El capítulo se organiza en tres secciones. La sección 2.1 presenta las gramáticas de concatenación de rango, su presentación intuitiva, sus definiciones formales y un ejemplo completo de reconocimiento. La sección 2.2 expone las propiedades de clausura del formalismo y su caracterización en términos de complejidad. La sección 2.3 revisa los antecedentes directos del trabajo: los estudios previos en la Facultad que abordan el SAT mediante formalismos de la teoría de lenguajes.

2.1. Gramáticas de concatenación de rango

Las gramáticas de concatenación de rango (RCG) [3] son un formalismo propuesto en 1998 por Pierre Boullier, un investigador en el campo de la lingüística computacional. El objetivo inicial del formalismo era proporcionar un modelo más expresivo que las gramáticas libres del contexto para describir ciertos fenómenos del lenguaje natural, como los números chinos y el orden variable de algunas palabras en alemán [4].

Antes de entrar a la presentación del formalismo, es necesario precisar una convención terminológica que rige a lo largo de todo el documento. El formalismo

que se presenta en este capítulo corresponde, en la literatura, a las **gramáticas de concatenación de rango positivas** (PRCG), una de las dos variantes del formalismo general de Boullier. La otra variante, las gramáticas de concatenación de rango con negación, admite cláusulas con predicados negados y no se utiliza en este trabajo. La restricción a la variante positiva es importante porque, como se establece más adelante, la clase de lenguajes reconocidos por una PRCG coincide exactamente con la clase *PTIME*, y los resultados de los capítulos posteriores se apoyan en dicha caracterización. Para abreviar, a lo largo del documento se adopta el nombre **gramática de concatenación de rango** (RCG) para referirse, salvo mención explícita, a la variante positiva.

La sección se organiza en tres partes. Primero se introducen de manera intuitiva los elementos de las gramáticas de concatenación de rango. Después se presentan las definiciones formales. Por último se presenta un ejemplo completo de reconocimiento por una RCG.

2.1.1. Presentación intuitiva

En esta subsección se presentan las nociones de predicado, argumento, sustitución de rango y derivación, que sirven de base para las definiciones formales de la siguiente subsección.

A los no terminales de una RCG se les llama **predicados**. Cada predicado tiene un conjunto de argumentos. A la cantidad de argumentos de un predicado se le denomina **aridad**. Por ejemplo, $A(X, Y)$ denota el predicado A con argumentos X e Y ; en este caso, la aridad de A es 2.

Cada argumento de un predicado puede estar formado por variables y terminales. Por convenio, las variables se denotan por letras mayúsculas del final del alfabeto, y los terminales con letras minúsculas. Con este convenio, el predicado $B(aX, XY, abZ)$ tiene aridad 3. El primer argumento está formado por el terminal a y la variable X . El segundo argumento está formado por la concatenación de las variables X e Y . El tercer argumento está formado por la concatenación de los terminales a y b y la variable Z .

Cada predicado reconoce un vector de cadenas cuya dimensión es la aridad del predicado. Cada cadena del vector se asocia a un argumento del predicado.

Por ejemplo, si al predicado $A(X, Y)$ se le asocia el vector $[abc, d]$, al primer argumento de A se le asigna la cadena abc y al segundo la cadena d . Esto significa que $X = abc$ y que $Y = d$.

De manera similar, si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[aa, de, abcc]$, al primer argumento de B se le asigna la cadena aa , al segundo la cadena de y al tercero la cadena $abcc$. Es decir, $aX = aa$, $XY = de$ y $abZ = abcc$. Como X es una variable y se tiene que $aX = aa$, el único valor que puede tomar X es $X = a$. En el tercer argumento ocurre algo similar: como $abZ = abcc$, para que se cumpla la igualdad el único valor posible para Z es $Z = cc$. En el segundo argumento, las variables X e Y deben tomar valores tales que su concatenación sea de ; esto se puede hacer de tres formas: $X = d$, $Y = e$; $X = de$, $Y = \varepsilon$; y $X = \varepsilon$, $Y = de$.

Si al predicado $B(aX, XY, abZ)$ se le asigna el vector $[bb, de, abcc]$, entonces al tenerse $aX = bb$, X no puede tomar ningún valor: el primer carácter b de la cadena no coincide con el terminal a que encabeza el argumento aX .

A las producciones de esta gramática se les denomina **cláusulas**. La parte izquierda de la producción siempre está formada por un único predicado. La parte derecha puede contener cualquier cantidad de predicados con sus respectivos argumentos, o la cadena vacía. Un ejemplo de cláusula es

$$A(XYZ, W) \rightarrow B(X) C(XY, Z) D(W).$$

La regla anterior tiene el siguiente significado: el predicado A recibe un vector de dimensión 2; a partir de las cadenas del vector, se asignan valores a las variables X , Y , Z y W , y con esos valores se construyen los vectores que reciben los predicados B , C y D .

El proceso se ilustra con un ejemplo en el que el predicado A recibe el vector de cadenas $[abc, d]$. El primer paso es asignar las cadenas del vector a los argumentos del predicado. El primer argumento de A es abc y el segundo es d . Como el primer argumento de A está definido como XYZ y el segundo como W , debe cumplirse que $XYZ = abc$ y $W = d$. Esto significa que las variables X , Y y Z deben tomar los valores posibles cuya concatenación sea abc , mientras que W solo puede tomar el valor d . En el cuadro 2.1 se muestran algunas asignaciones posibles para las variables X , Y , Z y W a partir del vector de entrada.

X	Y	Z	W
ε	ε	abc	d
ε	a	bc	d
ε	ab	c	d
ε	abc	ε	d
a	ε	bc	d
a	b	c	d
a	bc	ε	d
ab	ε	c	d
ab	c	ε	d
abc	ε	ε	d

Cuadro 2.1: Asignaciones posibles para las variables X , Y , Z y W a partir del vector $[abc, d]$.

Para continuar con el ejemplo, se supone que los valores de X , Y y Z son $X = ab$, $Y = \varepsilon$ y $Z = c$. A partir de esta asignación se construyen los vectores con los que se evalúan los predicados del lado derecho $B(X) C(XY, Z) D(W)$. Como $X = ab$, $Y = \varepsilon$, $Z = c$ y $W = d$, el lado derecho se evalúa con los vectores $[ab]$, $[ab, c]$ y $[d]$, y el resultado es la derivación $B(ab) C(ab, c) D(d)$.

El proceso de asignar los vectores a los argumentos, identificar los valores de las variables e instanciar las partes derechas se repite en cada uno de los predicados del lado derecho de la cláusula.

Existe otro tipo de producciones, denominadas cláusulas terminales, en las que un predicado deriva en ε para determinados valores de sus argumentos. Por ejemplo,

$$B(ab) \rightarrow \varepsilon, \quad C(ab, c) \rightarrow \varepsilon, \quad D(d) \rightarrow \varepsilon.$$

La forma general de estas producciones es $A(X_1, \dots, X_n) \rightarrow \varepsilon$.

Un predicado reconoce un vector de cadenas si existe una asignación de sus argumentos a valores de las variables y una secuencia de derivaciones que termina en la cadena vacía. Por ejemplo, en el caso de $B(ab) \rightarrow \varepsilon$, el predicado B reconoce la cadena ab .

A modo de ilustración, se considera el siguiente conjunto de producciones:

1. $A(XYZ, W) \rightarrow B(X) C(XY, Z) D(W)$,
2. $B(ab) \rightarrow \varepsilon$,
3. $C(ab, c) \rightarrow \varepsilon$,
4. $D(d) \rightarrow \varepsilon$.

Con estas producciones, el predicado A reconoce el vector $[abc, d]$: con la asignación $X = ab, Y = \varepsilon, Z = c$ y $W = d$, el predicado A deriva en $B(ab) C(ab, c) D(d)$, y cada uno de los predicados $B(ab)$, $C(ab, c)$ y $D(d)$ deriva en la cadena vacía.

Un predicado no reconoce un vector de cadenas cuando para ninguna asignación de variables se deriva en la cadena vacía.

Presentadas estas nociones, a continuación se introducen las definiciones formales de las gramáticas de concatenación de rango.

2.1.2. Definiciones formales

Definición 2.1. Un **rango** es una tupla (i, j) que representa un intervalo de posiciones en una cadena, donde i y j son enteros no negativos tales que $i \leq j$.

Por ejemplo, con índices indexados en 0, para la cadena $abcd$ el rango $(1, 3)$ representa la subcadena bc .

Definición 2.2. Una **gramática de concatenación de rango** se define como una 5-tupla

$$G = (N, T, V, P, S),$$

donde:

- N es un conjunto finito de **predicados** o símbolos no terminales; cada predicado tiene una aridad, que indica la dimensión del vector de cadenas que reconoce;
- T es un conjunto finito de **símbolos terminales**;
- V es un conjunto finito de **variables**;

- P es un conjunto finito de **cláusulas** de la forma

$$A(x_1, x_2, \dots, x_k) \rightarrow B_1(y_{1,1}, y_{1,2}, \dots, y_{1,m_1}) \dots B_n(y_{n,1}, y_{n,2}, \dots, y_{n,m_n}),$$

donde $A, B_i \in N$, $x_i, y_{i,j} \in (V \cup T)^*$, y k es la aridad de A ;

- $S \in N$ es el **predicado inicial** de la gramática, que siempre tiene aridad 1.

Por ejemplo, a continuación se muestra una gramática de concatenación de rango que reconoce el lenguaje $L_{\text{copy}}^3 = \{w^3 \mid w \in \{a, b, c\}^*\}$:

$$G_{\text{copy}}^3 = (N, T, V, P, S),$$

donde $N = \{A, S\}$, $T = \{a, b, c\}$, $V = \{X, Y, Z\}$, el símbolo inicial es S y el conjunto de cláusulas P es el siguiente:

1. $S(XYZ) \rightarrow A(X, Y, Z)$,
2. $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$,
3. $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$,
4. $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$,
5. $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$.

A diferencia de las gramáticas presentadas en el capítulo anterior, que se definen a partir de un mecanismo de generación de cadenas, las RCG se formulan directamente como un mecanismo de reconocimiento. Su funcionamiento consiste en determinar si una cadena pertenece o no al lenguaje.

Definición 2.3. Una **sustitución de rango** es un mecanismo que reemplaza una variable por un rango de la cadena, respetando la estructura del argumento que se asocia a la cadena que se reconoce.

Por ejemplo, dado el predicado $A(Xa)$ con $X \in V$ y $a \in T$, la estructura del argumento de A es una variable X seguida del terminal a . Si el predicado A recibe la cadena baa , entonces X se puede asociar con la subcadena ba de la cadena original: si $X = ba$, entonces $Xa = baa$. La variable X no puede tomar el valor baa , porque ningún carácter de la cadena de entrada coincide con el terminal a al final del argumento. La variable X tampoco puede tomar el valor b , porque con ese valor el argumento Xa no cubre la cadena completa.

Definición 2.4. Las **gramáticas de concatenación de rango simple** (SRCG) son un subconjunto de las RCG que restringen la forma de las cláusulas. Una RCG G es **simple** si los argumentos del lado derecho de cada cláusula son variables distintas, y todas estas variables aparecen una sola vez en los argumentos del lado izquierdo.

Las SRCG son equivalentes a los sistemas lineales de reescritura libres del contexto (LCFRS) introducidos en [9]. Este subconjunto se utiliza en [1] para describir el orden de las variables de una fórmula booleana en una variante del SAT.

Una vez introducidas las definiciones formales, a continuación se presenta un ejemplo completo de reconocimiento de una cadena por una RCG, aplicando los mecanismos descritos en esta sección.

2.1.3. Ejemplo de reconocimiento

La presentación intuitiva describió cómo se asocian los elementos de un vector a los argumentos de un predicado y cómo se construyen los vectores que reciben los predicados del lado derecho de una cláusula. En esta subsección se aplica ese proceso a un caso completo: el reconocimiento de la cadena $abcabcabc$ por la gramática G_{copy}^3 presentada anteriormente.

La cadena $abcabcabc$ se reconoce por G_{copy}^3 mediante la siguiente secuencia de derivaciones:

$$S(abcabcabc) \rightarrow A(abc, abc, abc) \rightarrow A(bc, bc, bc) \rightarrow A(c, c, c) \rightarrow A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon.$$

A continuación se describen los pasos.

- **Primer paso:** en la primera cláusula $S(XYZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = abc, Y = abc$ y $Z = abc$. De esta forma se deriva en el predicado $A(abc, abc, abc)$.
- **Segundo paso:** en la segunda cláusula $A(aX, aY, aZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = bc, Y = bc$ y $Z = bc$. Con estos valores se deriva en el predicado $A(bc, bc, bc)$.
- **Tercer paso:** en la tercera cláusula $A(bX, bY, bZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = c, Y = c$ y $Z = c$. Con estos valores se deriva en el predicado $A(c, c, c)$.
- **Cuarto paso:** en la cuarta cláusula $A(cX, cY, cZ) \rightarrow A(X, Y, Z)$, la sustitución de rango asocia las variables X, Y, Z a los valores $X = \varepsilon, Y = \varepsilon$ y $Z = \varepsilon$. Con estos valores se deriva en el predicado $A(\varepsilon, \varepsilon, \varepsilon)$.
- **Quinto paso:** en la última cláusula $A(\varepsilon, \varepsilon, \varepsilon) \rightarrow \varepsilon$ se deriva en la cadena vacía, con lo que la cadena $abcabcabc$ queda reconocida.

Una vez presentado el proceso de derivación, en la siguiente sección se analizan las propiedades del formalismo: clausura bajo operaciones sobre conjuntos, poder expresivo y complejidad del problema de la palabra.

2.2. Propiedades de las RCG

Las RCG fueron diseñadas como un formalismo modular, en el sentido de que las operaciones de unión, intersección y complemento son cerradas para

este formalismo [3]. La ausencia de esta modularidad en las gramáticas libres del contexto fue una de las limitaciones que motivó la propuesta original de las RCG [3]. En esta sección se describen las propiedades de clausura que demuestran la modularidad del formalismo y la caracterización en términos de complejidad.

Teorema 2.1 (Clausura bajo operaciones de conjuntos). *Las RCG son cerradas bajo unión, intersección y complemento. La unión y la intersección de dos RCG da como resultado un formalismo que pertenece a las RCG, mientras que el formalismo resultante del complemento de una RCG es también una RCG [3].*

En cuanto al poder expresivo, las RCG son estrictamente más expresivas que las gramáticas libres del contexto. Por ejemplo, para todo $k \geq 2$, el lenguaje $L_{\text{copy}}^k = \{w^k \mid w \in \Sigma^*\}$ es dependiente del contexto y no libre del contexto [8], y se reconoce por una RCG. En particular, el lenguaje L_{copy}^3 se reconoce por la gramática G_{copy}^3 del ejemplo anterior.

Una vez presentadas las propiedades estructurales, a continuación se analiza la caracterización del formalismo en términos de complejidad computacional, que constituye el resultado que motiva el uso de las RCG en este trabajo.

2.2.1. Complejidad y caracterización en términos de *PTIME*

El siguiente resultado es el pilar que conecta a las RCG con la teoría de la complejidad y sobre el que se apoyan los resultados de los capítulos posteriores.

Teorema 2.2 (Caracterización en términos de *PTIME*). *Un lenguaje L se reconoce por una RCG en su variante positiva si y solo si L admite un algoritmo de reconocimiento que se ejecuta en tiempo polinomial [2, 5].*

La demostración del teorema 2.2 se realiza en [2, 5]. La dirección que afirma que todo lenguaje RCG está en *PTIME* se sigue de la existencia de un algoritmo de reconocimiento polinomial para el formalismo, que se describe a continuación.

El algoritmo de reconocimiento estándar para las RCG utiliza un proceso de memorización sobre las cadenas asignadas a los argumentos de los predicados [3]. La cantidad máxima de rangos de una cadena de longitud n es n^2 , y la máxima aridad de un predicado es constante, por lo que el proceso de memorización emplea una cantidad polinomial de estados. La complejidad del algoritmo es $O(|P|n^{2h(l+1)})$, donde h es la máxima aridad de un predicado, l es la máxima cantidad de predicados en el lado derecho de una cláusula y n es la longitud de la cadena que se reconoce. El teorema establece, además, la dirección inversa: todo lenguaje en *PTIME* admite una RCG que lo reconoce.

El teorema 2.2 es el resultado que motiva la elección de las RCG como formalismo de base para este trabajo. El alcance exacto de la caracterización y sus consecuencias para la relación entre formalismos gramaticales y problemas de la clase *NP* son el objeto central de los capítulos posteriores.

Una vez establecida la caracterización de las RCG como formalismo que reconoce exactamente la clase *PTIME*, en la siguiente sección se revisan los antecedentes directos del trabajo.

2.3. Antecedentes en el estudio del SAT mediante formalismos de la teoría de lenguajes

El estudio del problema de la satisfacibilidad booleana mediante formalismos de la teoría de lenguajes cuenta con antecedentes directos en la Facultad de Matemática y Computación de la Universidad de La Habana. En esta sección se revisan los tres trabajos previos que constituyen el punto de partida del presente trabajo.

En [6] se propone una estrategia para resolver instancias del SAT mediante gramáticas libres del contexto. La estrategia consiste en tres pasos: asumir que todas las variables de la fórmula son distintas; construir un autómata finito que reconozca las cadenas de 0 y 1 que hacen verdadera la fórmula bajo ese supuesto; e intersectar el lenguaje regular obtenido con un lenguaje libre del contexto que obligue a que todas las instancias de una misma variable tomen el mismo valor. La intersección resultante es un lenguaje libre del contexto sobre el que el problema del vacío se decide en tiempo polinomial, con lo que el procedimiento completo opera en complejidad $O(n^3)$ para ciertas subclases del SAT.

En [1] se extiende la idea anterior al emplear gramáticas de concatenación de rango simple en lugar de gramáticas libres del contexto. El autómata booleano asociado a la fórmula se intersecta con una SRCG que modela el orden de las variables, lo que amplía el conjunto de fórmulas booleanas que admiten solución mediante esta estrategia.

En [7] se propone una estrategia diferente: en lugar de resolver el problema del vacío para el lenguaje de las interpretaciones que hacen verdadera a una fórmula dada, se construye el lenguaje de todas las fórmulas booleanas satisfacibles y se decide la satisfacibilidad mediante el problema de la palabra. Como parte de dicho trabajo se propone una gramática que reconocería el lenguaje de las fórmulas satisfacibles, y a partir de ella se derivan afirmaciones sobre la expresividad del formalismo con respecto a la clase NP .

Los tres trabajos anteriores conforman un programa de investigación acumulativo en la Facultad sobre la relación entre formalismos gramaticales y el SAT. El presente trabajo se inscribe en este programa y se propone examinar con precisión formal la clasificación de la gramática propuesta en [7] dentro de la jerarquía de formalismos considerada en este capítulo, analizar las consecuencias de dicha clasificación para los resultados derivados, y explorar construcciones de RCG para variantes polinomiales del SAT. Estos objetivos se desarrollan en los capítulos siguientes.

Bibliografía

- [1] Manuel Aguilera López. Problema de la satisfacibilidad booleana de concatenación de rango simple. In *Jornada Científica Estudiantil MatCom 2016*, pages 1–5, La Habana, Cuba, 2016. Facultad de Matemática y Computación, Universidad de La Habana.
- [2] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [3] Pierre Boullier. Proposal for a natural language processing syntactic backbone. Technical Report RR-3342, INRIA-Rocquencourt, France, 1998.
- [4] Pierre Boullier. Chinese numbers, MIX, scrambling, and range concatenation grammars. In *Proceedings of the 9th Conference of the European Chapter of the Association for Computational Linguistics (EACL '99)*, pages 53–60, Bergen, Norway, 1999.
- [5] Pierre Boullier. Range concatenation grammars. In *Proceedings of the Sixth International Workshop on Parsing Technologies (IWPT 2000)*, pages 53–64, Trento, Italy, 2000. Association for Computational Linguistics.
- [6] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [7] Raudel Alejandro Gómez Molina. Una aproximación al lenguaje de todas las fórmulas booleanas satisfacibles mediante gramáticas de concatenación de rango. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, February 2025.
- [8] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [9] K. Vijay-Shanker, David J. Weir, and Aravind K. Joshi. Characterizing structural descriptions produced by various grammatical formalisms. In

25th Annual Meeting of the Association for Computational Linguistics, pages 104–111, Stanford, California, USA, 1987. Association for Computational Linguistics.